

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development

(25CS1302E) Name: Valaboju Akshitha Roll No.: 2520080003 CO Mapping: CO2

SKILLING SESSION 7: VECTOR DATABASES – FOUNDATIONS

Aim

To understand why a traditional RDBMS index fails for similarity search (curse of dimensionality), what embeddings are, how cosine similarity, dot product and Euclidean distance differ, and how exact search compares with approximate nearest neighbour (ANN) indexes HNSW and IVF.

1. Curse of Dimensionality

A traditional RDBMS B-tree is designed for ordered or range-based lookups. For high-dimensional vector data, the nearest and farthest points become increasingly similar in distance. This is known as the curse of dimensionality, making it difficult for range-based indexes to prune the search space effectively.

Demonstration:

```
for dims in (2, 10, 100, 1000):
    points = rng.random((2000, dims))
    query = rng.random(dims)
    d = np.linalg.norm(points - query, axis=1)
    print(d.min(), d.max(), d.max() / d.min())
```

Dimensions	Nearest	Farthest	Farthest / Nearest
2	0.021	0.930	44.81
10	0.434	1.988	4.58
100	3.173	4.759	1.50
1000	12.124	13.663	1.13

As dimensions grow, the ratio tends toward 1, so the distinction between close and far points loses meaning.

2. Text Embeddings

Embeddings are dense vectors that represent the meaning or features of data. For the Task Hub example, task titles are converted into vectors so that semantically related tasks can be retrieved using vector similarity.

Embedding and query example:

```
tfidf = TfidfVectorizer(stop_words="english")
svd = TruncatedSVD(n_components=3, random_state=0)
doc_vectors = svd.fit_transform(tfidf.fit_transform(docs))
doc_vectors /= np.linalg.norm(doc_vectors, axis=1, keepdims=True)

query = "security issue with user login token"
q_vec = svd.transform(tfidf.transform([query]))[0]
q_vec /= np.linalg.norm(q_vec)
scores = doc_vectors @ q_vec
```

The source demonstrates LSA embeddings using TF-IDF followed by SVD. It also notes that production systems can use neural embedding models such as Sentence Transformers or an embedding API.

Rank	Cosine	Retrieved task
1	1.000	Fix login bug in the authentication service
2	1.000	Hash user passwords and secure the login token
3	0.986	Add JWT token authentication to the login API

3. Similarity Metrics

Three similarity/distance measures are demonstrated: cosine similarity, dot product, and Euclidean distance.

Pair	Cosine	Dot Product	Euclidean
a, b	1.000	28.0	3.742
a, c	0.714	10.0	2.828

Cosine: measures direction only and is useful for text embeddings. **Dot product:** considers direction and magnitude; for normalized vectors it equals cosine similarity. **Euclidean:** measures straight-line distance and is useful when absolute magnitude matters.

4. Exact Search vs Approximate Nearest Neighbour (ANN)

The experiment compares exact brute-force k-nearest-neighbour search with two ANN index families: IVF and HNSW, using 100,000 vectors of 128 dimensions and k = 10.

FAISS index construction:

```
exact = faiss.IndexFlatL2(DIM)
exact.add(data)

quantizer = faiss.IndexFlatL2(DIM)
ivf = faiss.IndexIVFFlat(quantizer, DIM, 256)
ivf.train(data)
ivf.add(data)

hns = faiss.IndexHNSWFlat(DIM, 32)
hns.hns.efConstruction = 80
hns.add(data)
```

Key comparison:

Index / Setting	Time (ms/query)	Recall@10
Flat (exact brute force)	2.958	1.000
IVF, nprobe=1	0.024	0.991
IVF, nprobe=8	0.173	1.000
IVF, nprobe=32	0.634	1.000
HNSW, efSearch=16	0.028	0.963
HNSW, efSearch=64	0.048	1.000
HNSW, efSearch=256	0.149	1.000

5. HNSW and IVF Trade-off

IVF divides the vector space into clusters using k-means and searches only selected clusters. The number of clusters examined is controlled by **nprobe**. HNSW uses a multi-layer proximity graph and controls the search effort using **efSearch**. Increasing nprobe or efSearch generally raises recall at the cost of query time.

In the experiment, HNSW used M = 32 links per node and had 4 layers. IVF used 256 clusters, with an average of 390 vectors per cluster.

Output / Result

The farthest-to-nearest distance ratio fell from about 45 in 2 dimensions to 1.13 in 1000 dimensions, showing why range-based indexes cannot effectively prune high-dimensional similarity searches. Dense text embeddings grouped security, database and UI tasks, and the login-token query retrieved the security-related tasks. Cosine similarity treated vectors a and b as identical in direction, while dot product and Euclidean distance also reflected magnitude or absolute separation. On 100,000 vectors, IVF and HNSW provided large speed improvements over brute-force search, with recall depending on nprobe and efSearch settings.